Link to Google Colab:
https://colab.research.google.com/drive/1j1aQEkqcnUxqVmgdrm6f_AixRQSacBSc?usp=sharing

`* I added # Geometric Visualization of all Calculations after our meeting; it looks long, but is for visualization only, not geometry altering.`

Suggestion #1:

- Do NOT move the apex along the class mean direction.

Instead:

- Move each apex from the class mean toward the centroid of all class means.

In CE24, the apex is defined as:

$$p_j = x_{0,j} + \mu_j f_j, \quad f_j = \frac{x - x_{0,j}}{\|x - x_{0,j}\|}$$

- $x_{0,j}$ = class mean
- x = barycenter (midpoint of class means)

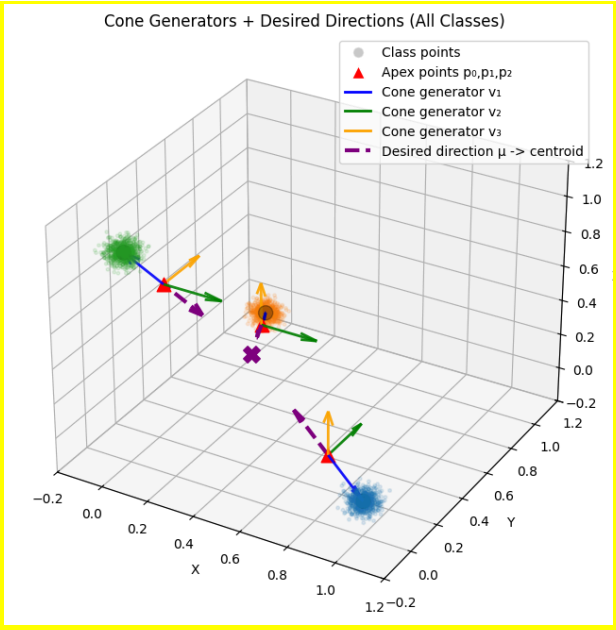
So:

- f_j points from the class toward the midpoint
- The apex p_j lies between the class and the other classes

My Fix (for Suggestion #1):

```
# CHANGES BASED ON CE24 Section 4, Page 24  ------------------ ADDITION #1 --------------------

# Computer Centroid
centroid = mus.mean(axis=0)

# Direction Vectors |  fj = (x-x0,j)/norm(x-x0,j)
directions = []

for k in range(Q):
    # vec = mus[k] - centroid # shouldn't it be the reverse? why is the this giving 100% accuracy
    vec = centroid - mus[k] # this doesn't seem to work, why is accuracy so low for the correct order?
    vec = vec / np.linalg.norm(vec)
    directions.append(vec)

directions = np.array(directions)

# Place Apex Points in front of the Class | Shift = x0,j + mu*fj
p0 = mus[0] + sep * directions[0]
p1 = mus[1] + sep * directions[1]
p2 = mus[2] + sep * directions[2]
```

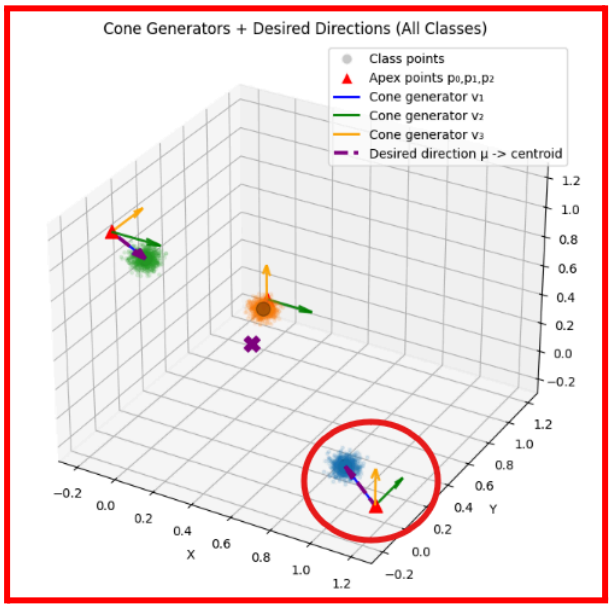
Results (for Suggestion #1):



*** As you can see in the plot above, the red triangles (p0, p1, p2) now lie on the vector between the class mean and the centroid.**

Issue/Questions Regarding Suggestion #1:

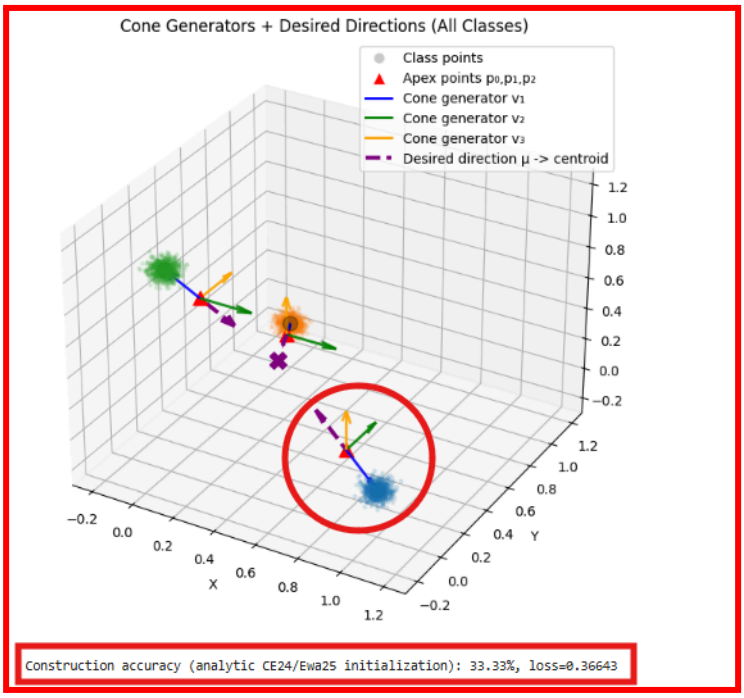
Using "`vec = mus[k] - centroid`" is essentially the backward cone direction, so adding with this vector pushes p0, p1, p2 away from the centroid as shown below…



However, it's weird because the flipped version gives 100% accuracy…

```
Construction accuracy (analytic CE24/Ewa25 initialization): 100.00%, loss=0.12888
```

And the correct version "`vec = centroid - mus[k]`," which pushes the apex point towards the forward cone (which I believe is correct), gives a low accuracy…



As of right now… except the issue of `centroid - mus[k]` vs. `mus[k] - centroid`, I believe I implemented your suggestion, so I am moving to Suggestion #2 and Suggestion #3 as I believe that could resolve this issue as "*The same direction used to move the apex must also be the central axis of the cone***."**

Suggestion #2 & Suggestion #3:

- **Compute the cone axis** using the **sum of the cone's generating vectors**, $a_j = v_{j,1} + v_{j,2} + v_{j,3}$, and compare its direction to the **collapse direction** $d_j = \mu_j$ - centroid.
- **Diagnose cone misalignment** by measuring the angle between the normalized vectors $a_j' = \frac{a_j}{||a_j||}$, $d_j' = \frac{d_j}{||d_j||}$, using the dot product. Derive θ to check if alignment (~ 0 degrees).
- Orient each truncation cone so that the *sum of its generator vectors* points in the same direction as the vector from the class mean toward the centroid.
- Compute the cone's current axis by summing its generator vectors, compare it to the class-mean-to-centroid direction, and if they are misaligned, add the difference between these two directions to *all* generators to tilt the cone without moving its apex.

My Fix (for Suggestion #2):

```
# ------------------------------------------------------------------------------------------------------------------------
# Cone Diagnostic to Check Alignment ------------------ ADDITION #2 ---------------------
# ------------------------------------------------------------------------------------------------------------------------

def cone_diagnostic(Vs, mus, centroid):
    print("\n================ CONE DIAGNOSTIC CHECK ================\n")

    for j, Vj in enumerate(Vs):
        print(f"--- Class {j} ---")

        # Generator vectors
        v1, v2, v3 = Vj[:, 0], Vj[:, 1], Vj[:, 2]

        # Cone axis (sum of generators)
        axis_raw = v1 + v2 + v3
        axis_norm = np.linalg.norm(axis_raw)
        axis = axis_raw / axis_norm

        # Collapse direction (backward cone direction)
        collapse_raw = mus[j] - centroid
        collapse_norm = np.linalg.norm(collapse_raw)
        collapse_dir = collapse_raw / collapse_norm

        # Dot product = cos(theta)
        cos_theta = np.dot(axis, collapse_dir)

        # Conversions
        cos_theta_clipped = np.clip(cos_theta, -1.0, 1.0)
        theta_rad = np.arccos(cos_theta_clipped)
        theta_deg = np.degrees(theta_rad)
```

What Additions I Made:

- Introduced an explicit **cone-axis diagnostic** by computing the cone's central axis as the **sum of its generating vectors**, $a_j = v_{j,1} + v_{j,2} + v_{j,3}$.
- Defined the **collapse direction** for each class as the vector from the global centroid to the class mean, $d_j = \mu_j -$ centroid, consistent with the CE24 truncation-map interpretation.
- Measured **geometric alignment** between the cone axis and the collapse direction using the **dot product**, and reported the corresponding angle in both radians and degrees.

Results (for Suggestion #2):

```
================ CONE DIAGNOSTIC CHECK ================

--- Class 0 ---
Class 0 Mean = [1.00181583e+00 3.54486240e-04 1.34482779e-03]
Centroid = [0.3329101  0.33284283 0.33425062]

Generator vectors:
  v1 = [ 0.20448133 -0.10164012 -0.10176773]
  v2 = [0. 1. 0.]
  v3 = [0. 0. 1.]

Cone axis (sum of generators):
  a_j = v1 + v2 + v3 = [0.20448133 0.89835988 0.89823227]
  ||a_j|| = 1.286734
  â_j = a_j / ||a_j|| = [0.158915   0.69817064 0.69807147]

Collapse direction:
  d_j = μ_j - centroid = [ 0.66890573 -0.33248834 -0.33290579]
  ||d_j|| = 0.817808
  d̂_j = d_j / ||d_j|| = [ 0.81792532 -0.40656048 -0.40707093]

Alignment check:
  cos(θ_j) = (â_j , d̂_j) = -0.438033
  θ_j = 2.024205 rad  (115.98°)
  ❌ Cone points the WRONG way
```

- This diagnostic check concludes that the cones are misaligned, and need the generating vectors to be tilted.

IMPORTANT: Suggestion #2 **did not** change any geometry, all it did was check alignment, so as of right now, moving the apex point in the forward cone direction (`centroid - mus[k]`) gives low accuracy; moving the apex point in the backward cone direction, which should be wrong (`mus[k] - centroid`), gives **100% accuracy**.

My Fix (for Suggestion #3):

> *"The question is how can you change the vectors so the cone tilts. What you have to add to them is probably the difference between the current sum and the direction you want." - Patrícia's Suggestion*

```
# ------------------------------------------------------------------------------------------------------
# Tilting the Cone Generators ------------------ ADDITION #3 --------------------
# ------------------------------------------------------------------------------------------------------

def tilt_cone_generators(Vj, mu_j, centroid, eps=0.5):

    # Current cone axis
    axis = Vj.sum(axis=1)
    axis = axis / np.linalg.norm(axis)

    # Desired collapse direction (backward cone direction)
    desired = mu_j - centroid
    desired = desired / np.linalg.norm(desired)

    # Difference = correction direction
    delta = desired - axis

    # Tilt all generators
    Vj_tilted = Vj + eps * delta[:, None]

    return Vj_tilted
```

```
V1 = tilt_cone_generators(V1, mus[0], centroid, eps=0.5)
V2 = tilt_cone_generators(V2, mus[1], centroid, eps=0.5)
V3 = tilt_cone_generators(V3, mus[2], centroid, eps=0.5)

W_cum1 = pinv(V1) # cone 1
b_cum1 = -W_cum1 @ p0

W_cum2 = pinv(V2) # cone 2
b_cum2 = -W_cum2 @ p1

W_cum3 = pinv(V3) # cone 3
b_cum3 = -W_cum3 @ p2

P = np.column_stack([p0, p1, p2])   # shape (3,3)
Y = np.eye(3)

W_cum4 = Y @ pinv(P) # Final Linear Classifier
b_cum4 = np.zeros(3)

W_list = [W_cum1, W_cum2, W_cum3, W_cum4]
b_list = [b_cum1, b_cum2, b_cum3, b_cum4]

Ws, bs = cumulative_to_layerwise(W_list, b_list) # Convert cumulative -> layerwise (CE24 Eq. (5)-(6))

cone_diagnostic([V1, V2, V3], mus, centroid) # running cone diagnostic again
```

What Additions I Made:

- Computed the **current cone axis** as the normalized sum of the generator vectors **($\mathbf{a}_j$)**, and compared it to the desired CE24 collapse direction **($\mathbf{d}_j$).**
- Defined a **correction direction** as the difference between the desired collapse direction and the current cone axis, $\Delta_j = \mathbf{d}_j - \mathbf{a}_j$, following the idea that the cone should be tilted toward the direction it is missing.
- Tilted **all cone generators simultaneously** by adding a small multiple of this correction vector, $v_{j,k}^{\text{tilted}} = v_{j,k} + \varepsilon\Delta_j$, ensuring that the cone axis rotates toward the correct direction **without changing the apex**.
- Reconstructed the truncation maps using the tilted generator matrices and reran the **cone diagnostic**, verifying that the cone axis now aligns with the collapse direction as required.

Results (for Suggestion #3):

- Before Titling:

```
================ CONE DIAGNOSTIC CHECK ================

--- Class 0 ---
Class 0 Mean = [1.00181583e+00 3.54486240e-04 1.34482779e-03]
Centroid = [0.3329101  0.33284283 0.33425062]

Generator vectors:
  v1 = [ 0.20448133 -0.10164012 -0.10176773]
  v2 = [0. 1. 0.]
  v3 = [0. 0. 1.]

Cone axis (sum of generators):
  a_j = v1 + v2 + v3 = [0.20448133 0.89835988 0.89823227]
  ||a_j|| = 1.286734
  â_j = a_j / ||a_j|| = [0.158915   0.69817064 0.69807147]

Collapse direction:
  d_j = μ_j - centroid = [ 0.66890573 -0.33248834 -0.33290579]
  ||d_j|| = 0.817808
  d̂_j = d_j / ||d_j|| = [ 0.81792532 -0.40656048 -0.40707093]

Alignment check:
  cos(θ_j) = ⟨â_j , d̂_j⟩ = -0.438033
  θ_j = 2.024205 rad  (115.98°)
  ❌ Cone points the WRONG way

------------------------------------------------------
```

- After Titling:

```
================ CONE DIAGNOSTIC CHECK ================

--- Class 0 ---
Class 0 Mean = [1.00181583e+00 3.54486240e-04 1.34482779e-03]
Centroid = [0.3329101  0.33284283 0.33425062]

Generator vectors:
  v1 = [ 0.53398649 -0.65400568 -0.65433893]
  v2 = [ 0.32950516  0.44763444 -0.5525712 ]
  v3 = [ 0.32950516 -0.55236556  0.4474288 ]

Cone axis (sum of generators):
  a_j = v1 + v2 + v3 = [ 1.19299681 -0.75873681 -0.75948133]
  ||a_j|| = 1.604910
  â_j = a_j / ||a_j|| = [ 0.74334206 -0.47275984 -0.47322374]

Collapse direction:
  d_j = μ_j - centroid = [ 0.66890573 -0.33248834 -0.33290579]
  ||d_j|| = 0.817808
  d̂_j = d_j / ||d_j|| = [ 0.81792532 -0.40656048 -0.40707093]

Alignment check:
  cos(θ_j) = ⟨â_j , d̂_j⟩ = 0.992839
  θ_j = 0.119743 rad  (6.86°)
  ✅ Cone is well-aligned (CE24-style)

------------------------------------------------------
```

- As seen, tilting the cone with respect to $d_j = \mu_j - \text{centroid}$ (once again, I am confused if it is this direction or if the RHS expression should be flipped), results in the alignment metric, cos(theta) to be very close to 1, showing tilting was successful.

- **Additionally, in this addition, we actual redefine the geometry of the cone, so we have to rerun the model to check for accuracy, and this time the accuracy drops from 100% to**

```
Construction accuracy (analytic CE24/Ewa25 initialization): 33.33%, loss=0.44444
```

.

Final Summary:

- **Suggestion #1 (Apex placement):**
  - Move each cone apex from the class mean toward the centroid of all class means so that the apex lies between the target class and the other classes, consistent with the CE24 definition of the truncation map.
- **Result:**
  - Placing the apex using **centroid − μ** geometrically matches CE24 but yields low accuracy when the cone generators are misaligned, while using **μ − centroid** places the apex in the backward cone and gives high accuracy for the wrong geometric reason.
- **Suggestion #2 (Diagnostic, no geometry change):**
  - Compute the cone's central axis as the sum of its generating vectors and diagnose misalignment by measuring the angle between this axis and the collapse direction from the class mean to the centroid using the dot product.
- **Result:**
  - The diagnostic shows that all cones are **strongly misaligned ($\theta \approx 116°$)**, explaining why correct apex placement alone does not yield zero loss.
- **Suggestion #3 (Cone tilting / geometry correction):**
  - Tilt each cone by adding a small multiple of the difference between the desired collapse direction and the current cone axis to all generator vectors, rotating the cone without moving its apex.
- **Result:**
  - After tilting, the cone axes align with the collapse directions **($\cos\theta \approx 1$)**, confirming the geometric fix, but accuracy drops.

$d_j = \mu_j - \text{centroid}$ is the **backward cone direction** | $d_j = \text{centroid} - \mu_j$ is the **forward cone direction**